Recurrent Neural Networks

Module 3

Computational Graphs

- The computational graph is a directed graph that is used for expressing and evaluating mathematical expressions.
- These can be used for two different types of calculations:
  - Forward computation
  - Backward computation

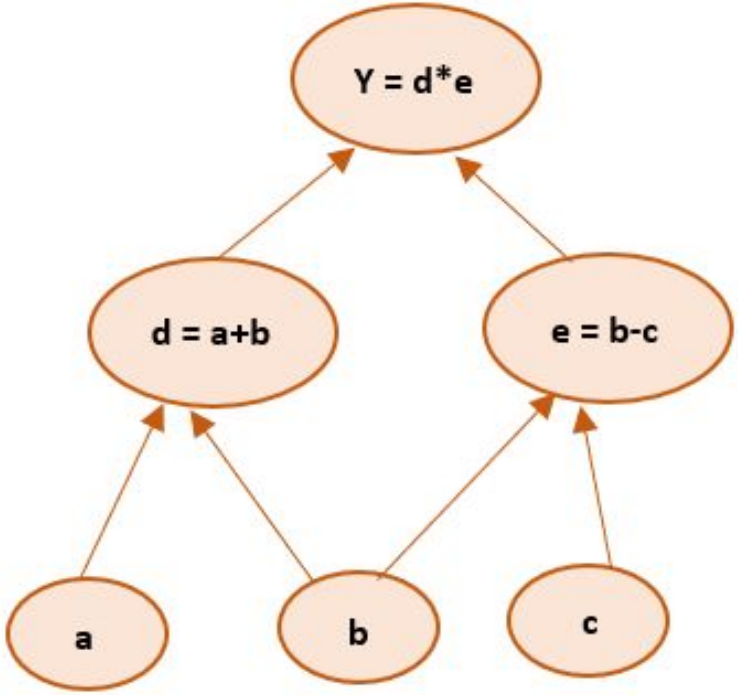
- A variable is represented by a node in a graph. It could be a scalar, vector, matrix, tensor, or even another type of variable.
- A function argument and data dependency are both represented by an edge. These are similar to node pointers.
- A simple function of one or more variables is called an operation. There is a set of operations that are permitted.

- Functions that are more complex than the operations in this set are represented by combining multiple operations.

Ex: Y=(a+b)*(b-c)

- For understanding: introduce two variables d and e such that every operation has an output variable.
  - d=a+b
  - e=b-c
  - Y=d*e
- Here, we have three operations, addition, subtraction, and multiplication.

- To create a computational graph, we create nodes, each of them has different operations along with input variables. The direction of the array shows the direction of input being applied to other nodes.

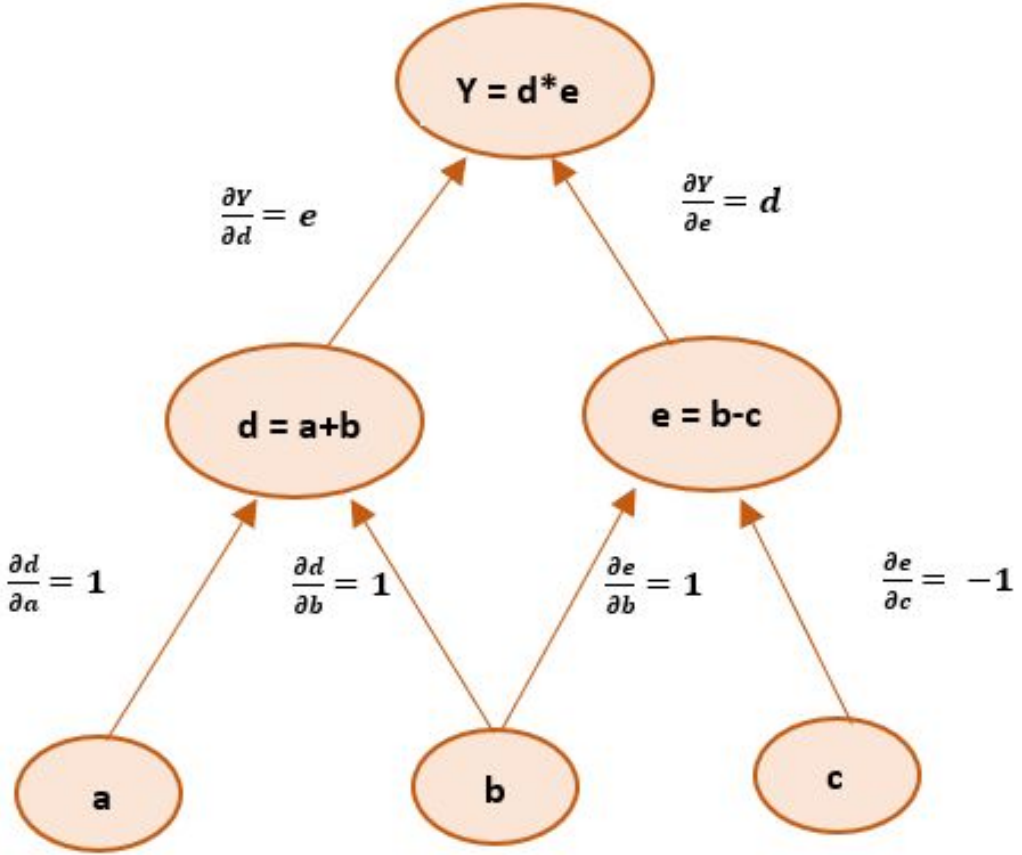


Computational Graphs in Deep Learning

- Computations of the neural network are organized in terms of a forward pass or forward propagation step in which we compute the output of the neural network, followed by a backward pass or backward propagation step, which we use to compute gradients/derivatives.
- Computation graphs explain why it is organized this way.

- To understand derivatives in a computational graph, the key is to understand how a change in one variable brings change on the variable that depends on it.
- If a directly affects c, then we want to know how it affects c.
- If we make a slight change in the value of a how does c change? We can term this as the partial derivative of c with respect to a.

Graph for backpropagation to get derivatives

- 

Follow chain rule to evaluate partial derivatives of final output variable with respect to input variables: a, b, and c. Therefore the derivatives can be given as :

$$\frac{\partial Y}{\partial a} = \frac{\partial Y}{\partial d} \times \frac{\partial d}{\partial a} = e \times 1 = e$$

$$\frac{\partial Y}{\partial b} = \frac{\partial Y}{\partial d} \times \frac{\partial d}{\partial b} = e \times 1 + d \times 1 = e + d$$

$$\frac{\partial Y}{\partial c} = \frac{\partial Y}{\partial e} \times \frac{\partial e}{\partial c} = d \times -1 = -d$$

Types of computational graphs:

- **Type 1:** Static Computational Graphs
- Involves two phases:-
  - Phase 1:- Make a plan for your architecture.
  - Phase 2:- To train the model and generate predictions, feed it a lot of data.
- The benefit of utilizing this graph is that it enables powerful offline graph optimization and scheduling. As a result, they should be faster than dynamic graphs in general.
- The drawback is that dealing with structured and even variable-sized data is unsightly.

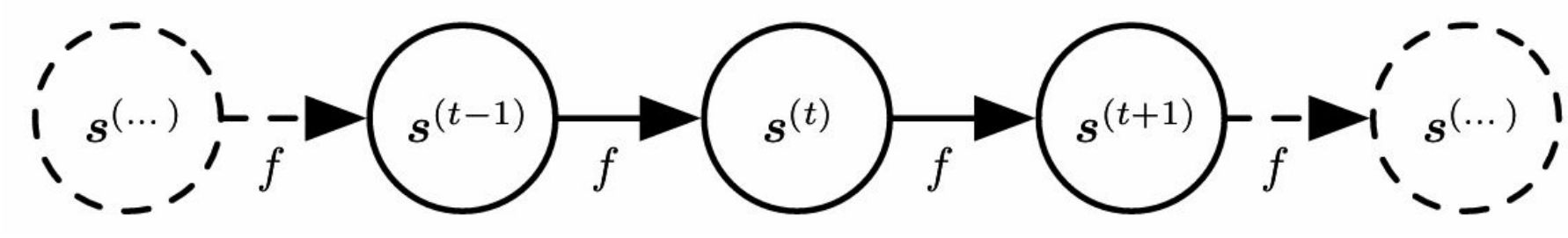
- **Type 2:** Dynamic Computational Graphs
- As the forward computation is performed, the graph is implicitly defined.
- This graph has the advantage of being more adaptable. The library is less intrusive and enables interleaved graph generation and evaluation.
- The forward computation is implemented in your preferred programming language, complete with all of its features and algorithms.
- Debugging dynamic graphs is simple. Because it permits line-by-line execution of the code and access to all variables, finding bugs in your code is considerably easier.
- The disadvantage of employing this graph is that there is limited time for graph optimization, and the effort may be wasted if the graph does not change.

Unfolding Computational Graphs

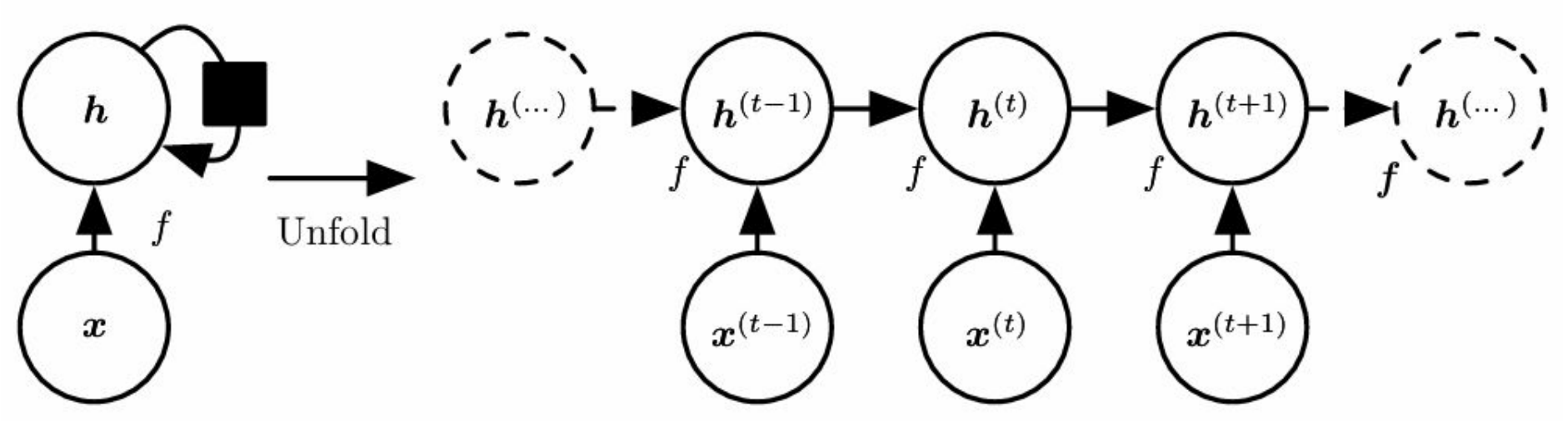
- We can unfold a recursive or recurrent computation into a computational graph that has a repetitive structure
  - Corresponding to a chain of events
- Unfolding this graph results in sharing of parameters across a deep network structure

Classical Dynamic System

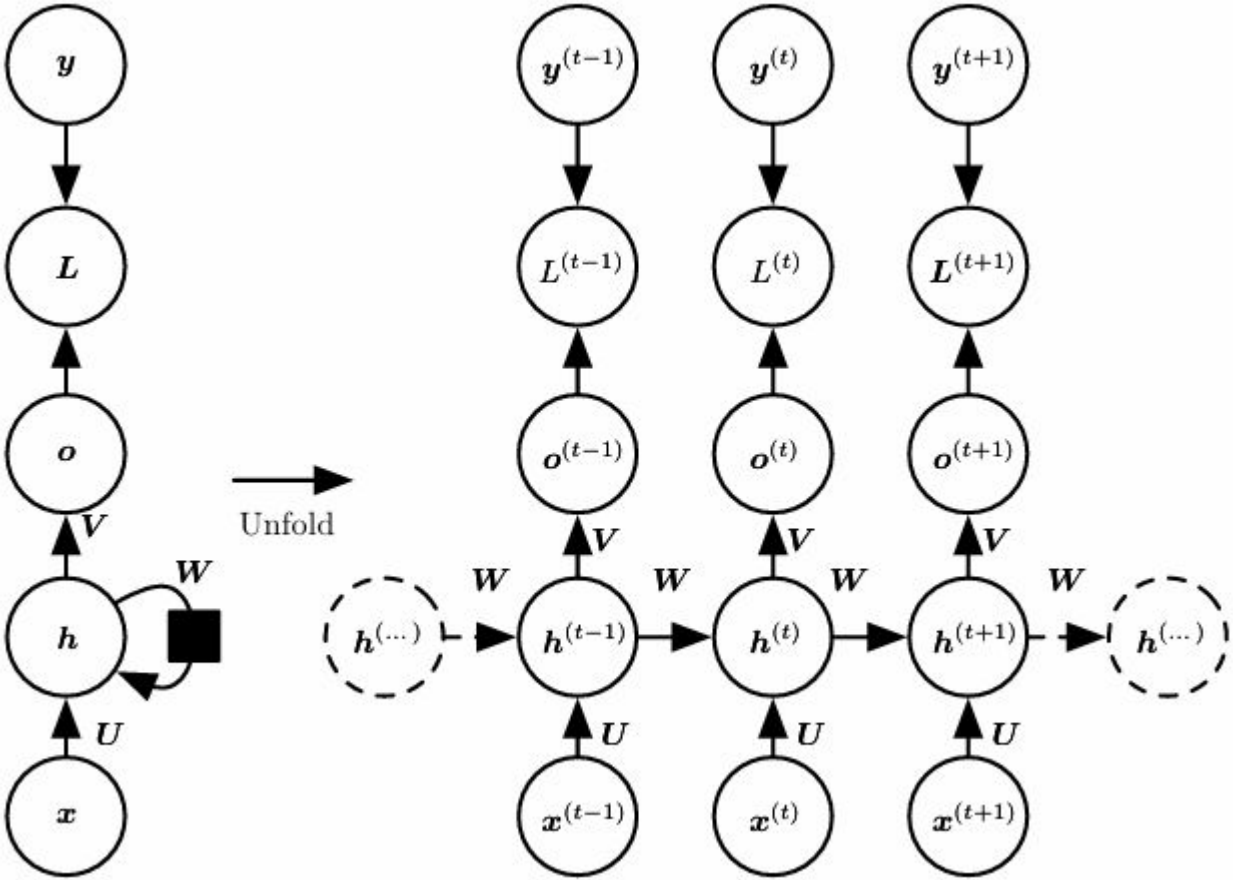
•



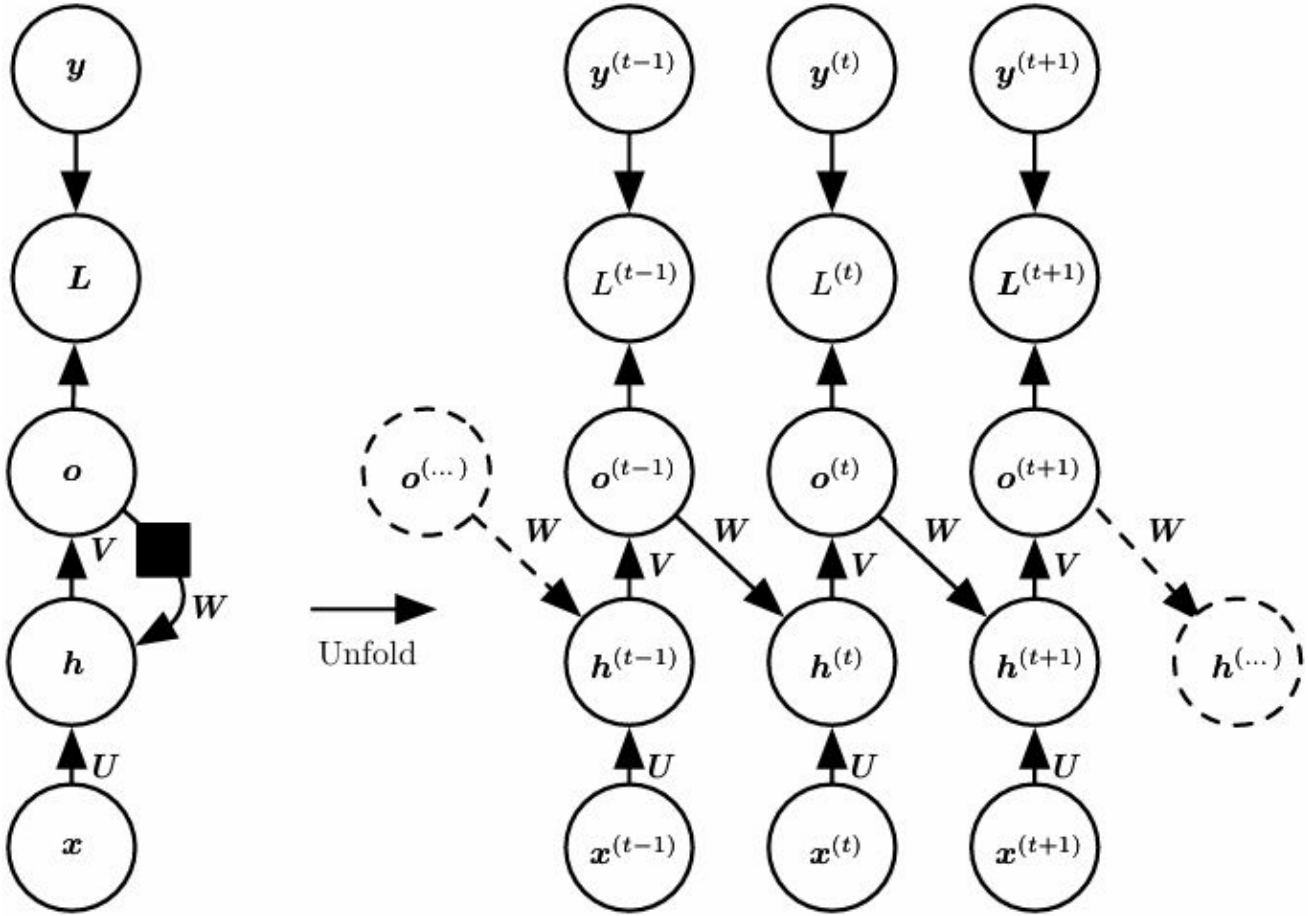
Unfolding Computational Graph



Recurrent Hidden Units

- 

Recurrence through only the Output

- 

Introduction to Recurrent Neural Networks

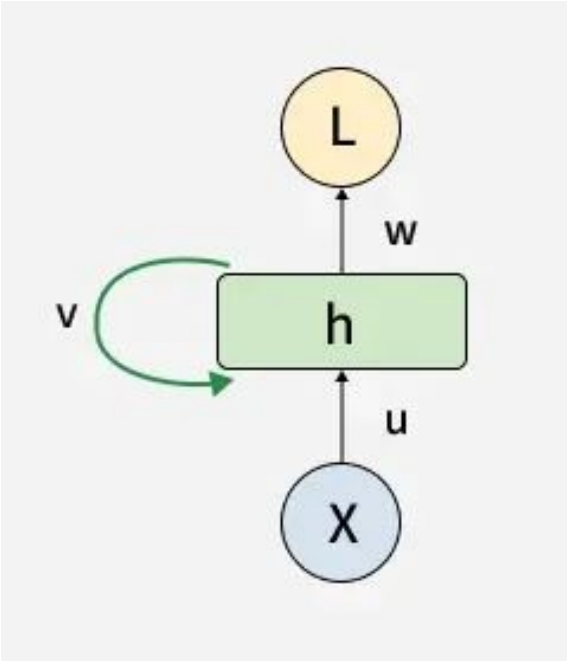
- Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps.
- They are especially effective for tasks where context and order matter.
  - Designed for sequential and temporal data
  - Maintains memory of past inputs
  - Widely used in NLP, forecasting and speech tasks

RNN with example

- Imagine reading a sentence and try to predict the next word, we don’t rely only on the current word but also remember the words that came before.
- RNNs work similarly by “remembering” past information i.e it considers all the earlier words to choose the most likely next word.
- This memory of previous steps helps the network understand context and make better predictions.

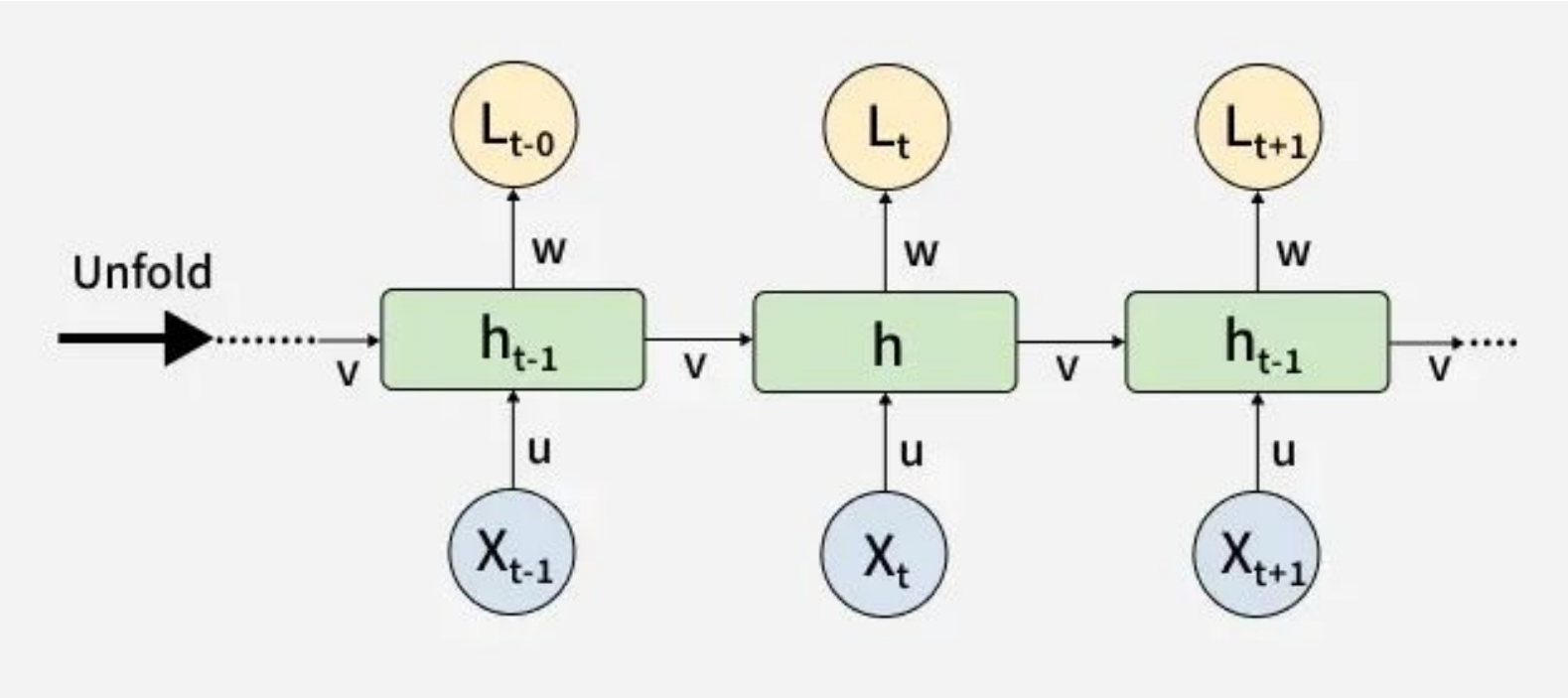
Key Components of RNNs

1. **Recurrent Neurons**
   - The fundamental processing unit in RNN is a Recurrent Unit.
   - They hold a hidden state that maintains information about previous inputs in a sequence.
   - Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.


L
w
v
h
u
X

2. RNN Unfolding

- RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps.
- During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.
- This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network’s weights enhancing the RNN’s ability to learn dependencies within sequential data.


Unfold
L_{t-0}
L_t
L_{t+1}
w
w
w
h_{t-1}
h
h_{t-1}
v
v
v
v
u
u
u
X_{t-1}
X_t
X_{t+1}

Recurrent Neural Network Architecture

- RNNs share similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output.
- Unlike traditional deep neural networks where each dense layer has distinct weight matrices.
- RNNs use shared weights across time steps, allowing them to remember information over sequences.

1. Hidden State Calculation:

- In RNNs the hidden state H_i is calculated for every input X_i to retain sequential dependencies.

$$h = \sigma(U.X + W.h_{t-1} + B)$$

- Here
  - h represents the current hidden state.
  - U and W are weight matrices.
  - B is the bias.

2. Output Calculation:

$$Y = O(V.h + C)$$

- The output Y is calculated by applying O an activation function to the weighted hidden state where V and C represent weights and bias.

3. Overall Function:

$$Y = f(X, h, W, U, V, B, C)$$

- This function defines the entire RNN operation where the state matrix S holds each element s_i representing the network's state at each time step i.

How does RNN work?

- At each time step RNNs process units with a fixed activation function.
- These units have an internal hidden state that acts as memory that retains information from previous time steps.
- This memory allows the network to store past knowledge and adapt based on new inputs.

Updating the Hidden State in RNNs

1. The current hidden state h_t depends on the previous state h_{t-1} and the current input x_t and is calculated using the following relations:

$$h_t = f(h_{t-1}, x_t)$$

- Where:
  - h_t is the current state
  - h_{t-1} is the previous state
  - x_t is the input at the current time step

2. Activation function application:

$$h_t = tanh(W_{hh}.h_{t-1} + W_{xh}.x_t)$$

- Here,
  - W_{hh} is the weight matrix for the recurrent neuron
  - W_{xh} is the weight matrix for the input neuron.

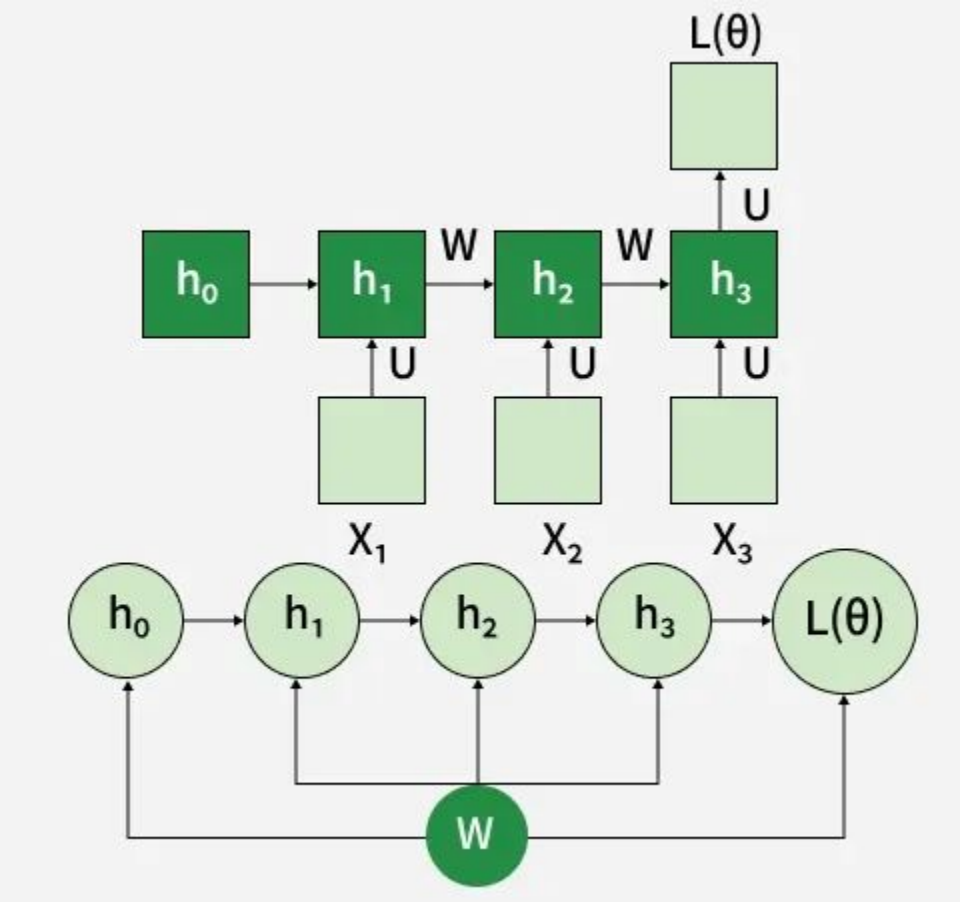
3. Output Calculation: $y_t = W_{hy}.h_t$

- where
  - y_t is the output and
  - W_{hy} is the weight at the output layer.

Since RNN works on sequential data here we use an updated backpropagation which is known as backpropagation through time to update these parameters.

Backpropagation Through Time

The loss function $L(\theta)$ depends on the final hidden state h_3 and each hidden state relies on preceding ones forming a sequential dependency chain:h_3 depends on h_2, h_2 on h_1 and h_1 on h_0.



1. **Simplified Gradient Calculation:** $\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \cdot \frac{\partial h_3}{\partial W}$

2. **Handling Dependencies in Layers:** Each hidden state is updated based on its dependencies:

$$h_3 = \sigma(W.h_2 + b)$$

The gradient is then calculated for each state, considering dependencies from previous hidden states.

3. **Gradient Calculation with Explicit and Implicit Parts:** The gradient is broken down into explicit and implicit parts summing up the indirect paths from each hidden state to the weights.

$$\frac{\partial h_3}{\partial W} = \frac{\partial h_3^+}{\partial W} + \frac{\partial h_3}{\partial h2} \cdot \frac{\partial h_2^+}{\partial W}$$

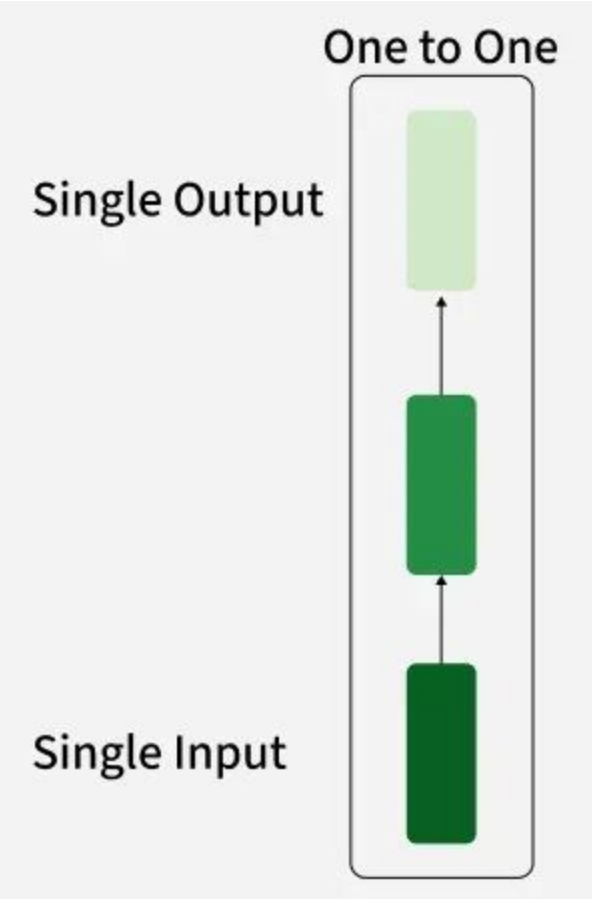
4. **Final Gradient Expression:** The final derivative of the loss function with respect to the weight matrix W is computed:

$$\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \cdot \sum_{k=1}^{3} \frac{\partial h_3}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}$$

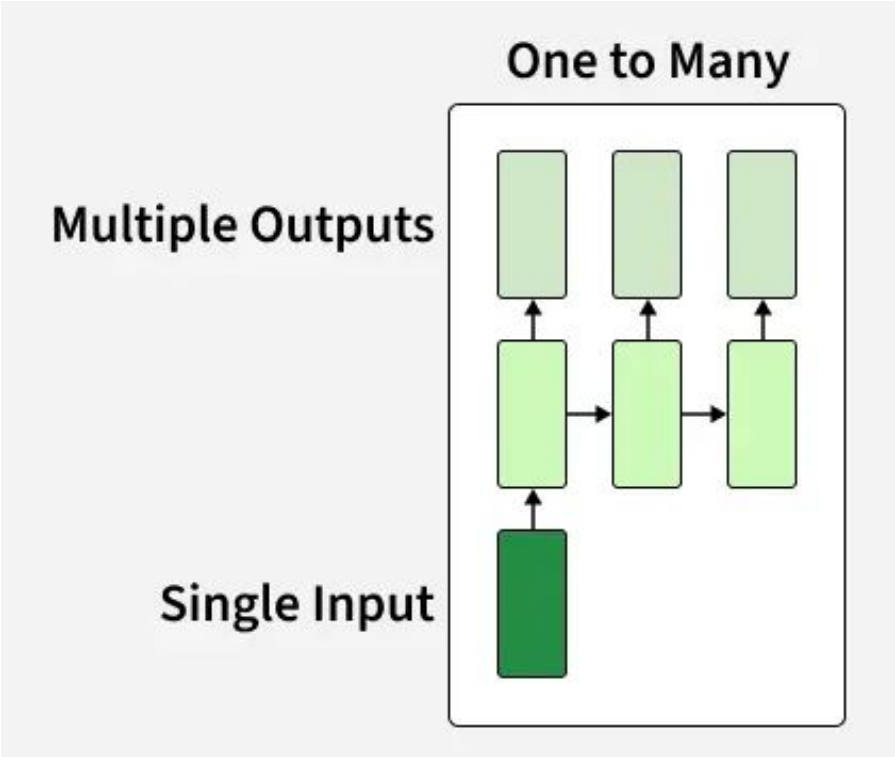
This iterative process is the essence of backpropagation through time.

Types Of Recurrent Neural Networks

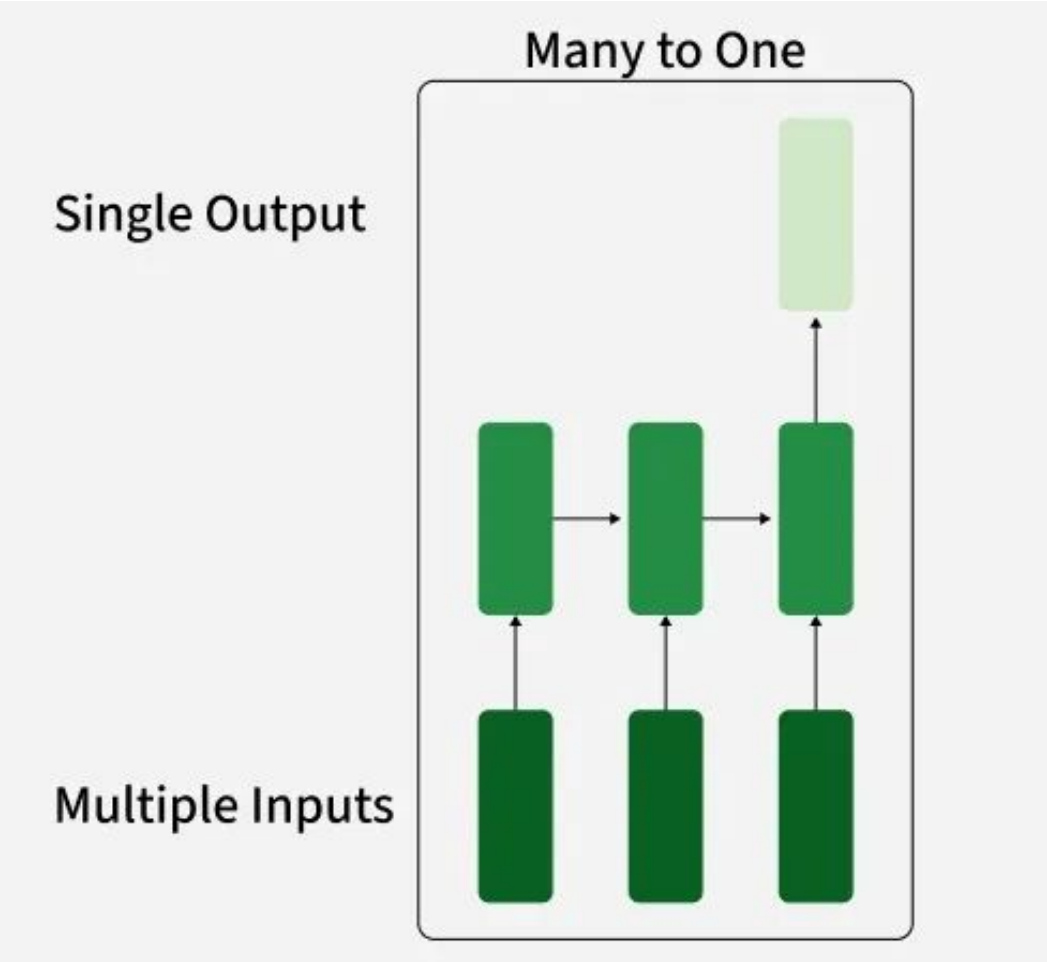
- **One-to-One RNN**
  - This is the simplest type of neural network architecture where there is a single input and a single output.
  - It is used for straightforward classification tasks such as binary classification where no sequential data is involved.


One to One
Single Output
Single Input

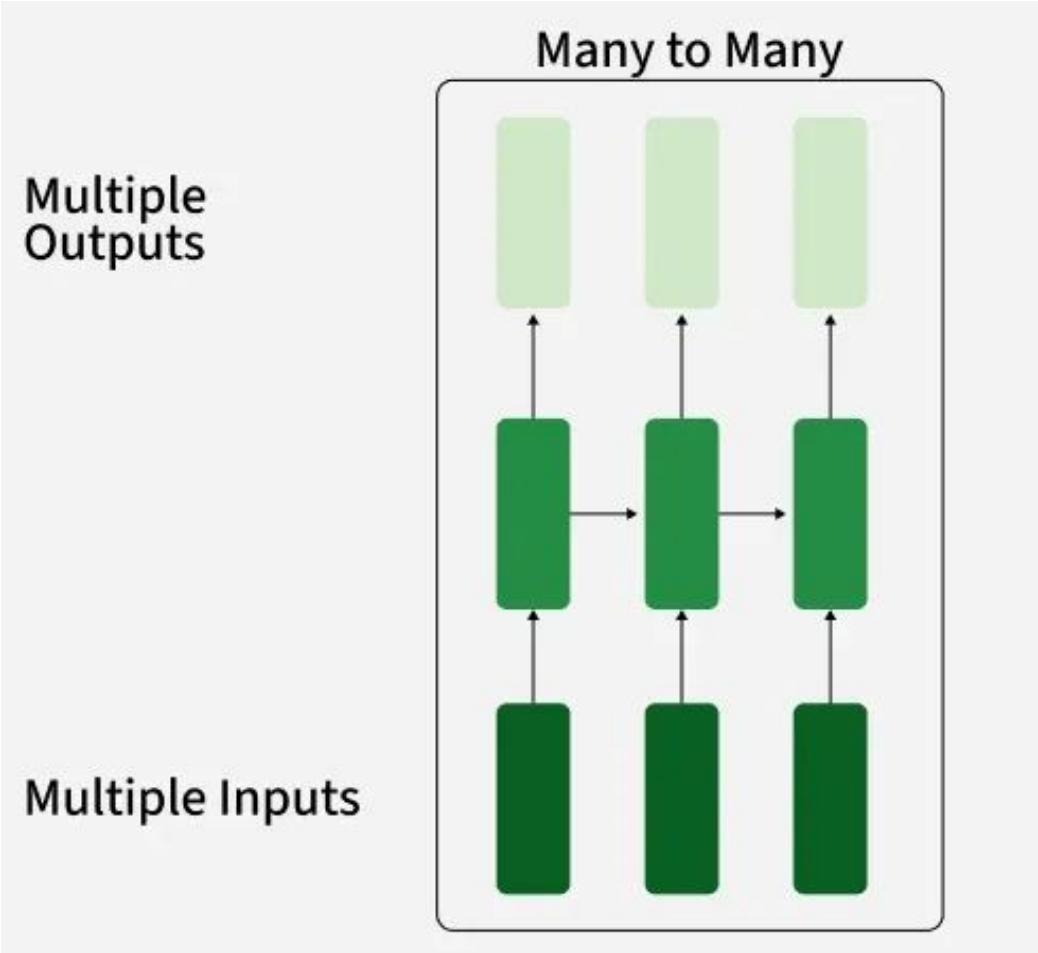
- **One-to-Many RNN**
  - In a One-to-Many RNN the network processes a single input to produce multiple outputs over time.
  - This is useful in tasks where one input triggers a sequence of predictions (outputs).
  - For example in image captioning a single image can be used as input to generate a sequence of words as a caption.


One to Many
Multiple Outputs
Single Input

- **Many-to-One RNN**
  - The Many-to-One RNN receives a sequence of inputs and generates a single output.
  - This type is useful when the overall context of the input sequence is needed to make one prediction.
  - In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.


Many to One
Single Output
Multiple Inputs

- **Many-to-Many RNN**
  - The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs.
  - In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.


Many to Many
Multiple Outputs
Multiple Inputs

Variants of RNNs

- There are several variations of RNNs, each designed to address specific challenges or optimize for certain tasks:
- **Vanilla RNN**
  - This RNN consists of a single hidden layer where weights are shared across time steps.
  - Vanilla RNNs are suitable for learning short-term dependencies but are limited by the vanishing gradient problem, which hampers long-sequence learning.

- **Bidirectional RNNs**
  - Bidirectional RNNs process inputs in both forward and backward directions, capturing both past and future context for each time step.
  - This architecture is ideal for tasks where the entire sequence is available, such as named entity recognition and question answering.

- **Long Short-Term Memory Networks (LSTMs)**
  - Long Short-Term Memory Networks (LSTMs) introduce a memory mechanism to overcome the vanishing gradient problem.
  - Each LSTM cell has three gates:
    - **Input Gate:** Controls how much new information is added to the cell state.
    - **Forget Gate:** Decides what past information is discarded.
    - **Output Gate:** Regulates what information is output at the current step. This selective memory enables LSTMs to handle long-term dependencies. Ideal for tasks where earlier context is critical.

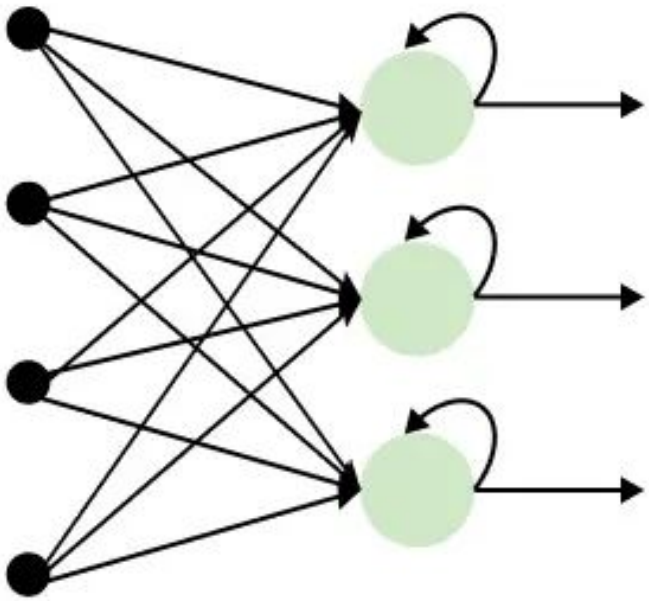
- **Gated Recurrent Units (GRUs)**
  - Simplifies LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism.
  - This design is computationally efficient, often performing similarly to LSTMs and is useful in tasks where simplicity and faster training are beneficial.

RNN Vs Feedforward Neural Networks

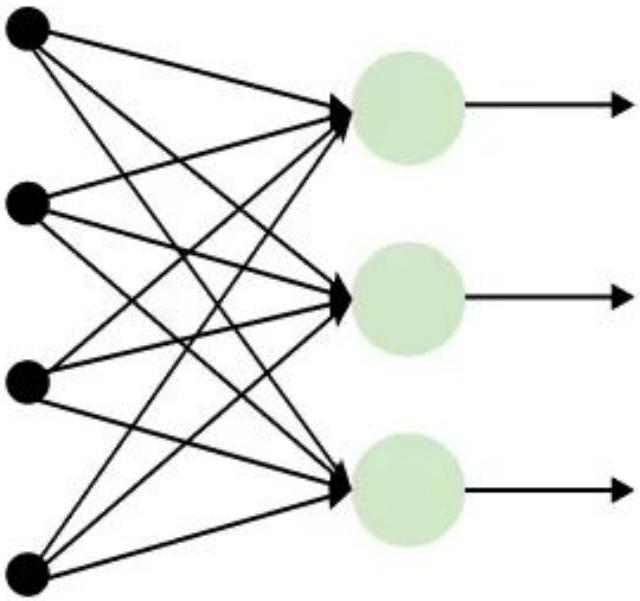
- Include feedback loops that pass information from previous steps.
- Maintain memory of past inputs through hidden states.
- Designed for sequential and time-dependent data.
- Effective for tasks where context matters, such as text and time-series analysis.

- Process data in a single direction, from input to output.
- Do not store information from previous inputs.
- Suitable for tasks with independent data, such as image classification.
- Perform poorly on sequential data due to the absence of memory.

(a) Recurrent Neural Network



(b) Feed-Forward Neural Network



Advantages

- **Sequential Memory:** RNNs retain information from previous inputs making them ideal for time-series predictions where past data is crucial.
- **Enhanced Pixel Neighborhoods:** RNNs can be combined with convolutional layers to capture extended pixel neighborhoods improving performance in image and video data processing.

Limitations

- Two main training challenges:
  - **Vanishing Gradient:** During backpropagation gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn long-term dependencies which is crucial for tasks like language translation.
  - **Exploding Gradient:** Sometimes gradients grow uncontrollably causing excessively large weight updates that destabilize training.
- These challenges can hinder the performance of standard RNNs on complex, long-sequence tasks.

Applications

- **Time-Series Prediction:** Forecasting tasks, such as stock market predictions and weather forecasting.
- **Natural Language Processing (NLP):** RNNs are fundamental in NLP tasks like language modeling, sentiment analysis and machine translation.
- **Speech Recognition:** Capture temporal patterns in speech data, aiding in speech-to-text and other audio-related applications.
- **Image and Video Processing:** When combined with convolutional layers, RNNs help analyze video sequences, facial expressions and gesture recognition.

Modeling Sequences Conditioned on Context with RNNs

- Many real-world problems require generating or predicting sequences based on some context.
- Examples:
  - Machine Translation: English → French sentence
  - Image Captioning: Image → “A dog playing with a ball”
  - Speech Recognition: Audio signal → text
  - Chatbots / Dialogue Systems
- **Key idea:**The output sequence depends on some additional input called context.

$$P(y_1, y_2, \ldots y_t \mid c)$$

- Where
  - c = context
  - y_t = output sequence
- Instead of modelling $P(y_1, y_2, \ldots y_t)$, we model the conditional probability.

Context

- Context is additional information used to generate the sequence.

Task	Context	Output
Translation	Source sentence	Target sentence
Image captioning	Image features	Caption
Question answering	Question	Answer sequence

Conditional Sequence Modeling

- Using Chain Rule:

$$P(y_1, y_2, \ldots, y_T \mid c) = \prod_{t=1}^{T} P(y_t \mid y_1, \ldots, y_{t-1}, c)$$

- Each output depends on previous outputs and context
- Ex: Translation:P(bonjour|hello)

RNN for Conditional Modeling

- We modify a standard RNN to include context c.
- Hidden state update:

$$h_t = f(W_h h_{t-1} + W_y y_{t-1} + W_c c)$$

- Output:

$$P(y_t|h_t) = Softmax(W_o h_t)$$

- Where
  - c = context vector
  - h_t = hidden state
- So the RNN remembers context during generation.

Approaches

1. Context as Additional Input
2. Context as Initial State

Context as Additional Input

- Context is fed at every time step.
- Hidden state: $h_t = f(W_h h_{t-1} + W_y y_{t-1} + W_c c)$
- Advantages
  - context always available
  - easy to implement
- Used in: language models with topic vectors

Context as Initial State

- Use context to initialize hidden state:

$$h_0 = f(W_c c)$$

- Then run normal RNN:

$$h_t = f(W_h h_{t-1} + W_x x_t)$$

- Advantages
  - simpler model
  - commonly used in encoder-decoder models

Variable-Length Sequences

Many sequence tasks have different input and output lengths.

Input sentence:

```
I love deep learning
```

Output sentence:

```
J'aime l'apprentissage profond
```

Input length ≠ Output length.

- Encoder-decoder models solve this by:
  - Encoding the input sequence into a context vector.
  - Decoding it into an output sequence of any length.

Encoder-Decoder Architecture

- Very important architecture.
- Used in: Machine translation, Text summarization, Dialogue systems
- Encoder
  - Reads input sequence: $x_1, x_2, \ldots x_t$
  - Produces context vector: $c = h_t$
- Decoder
  - Generates output sequence using context: $y_1, y_2, \ldots y_t$

Thus: Input → Encoder RNN → context c → Decoder RNN

Ex: Machine Translation

Input:

```
I love deep learning
```

Encoder converts sentence to context vector:

```
c
```

Decoder generates:

```
J' aime l'apprentissage profond
```

Each word predicted sequentially.

Training the Model

Training objective:

Maximize likelihood:

$$\log P(y_1, ..., y_T | c)$$

Loss function:

Cross-Entropy Loss

Training method:

Backpropagation Through Time (BPTT)

Teacher Forcing

- During training, the decoder receives the true previous word as input instead of the word predicted by the model.

Example: Machine Translation

Input sentence:

```
I love AI
```

Correct output sequence:

```
<start> J'aime IA <end>
```

With Teacher Forcing (Training)

Step	Decoder Input	Target Output
1	`<start>`	J'aime
2	**J'aime (true previous word)**	IA
3	**IA (true previous word)**	`<end>`

The **actual previous word from the training data** is used as input.

Teacher forcing

- Helps the model learn faster
- Prevents error accumulation during training
- Makes training more stable

Without teacher forcing:

- If the model makes a mistake early, that wrong word becomes the next input, and errors keep propagating through the sequence.

Teacher forcing avoids this during training by always feeding the correct previous word. This is how Seq2Seq models are trained.

Limitations

- Main issue
  - Single context vector bottleneck
  - For long sequences:
    - encoder compresses everything into one vector
    - information loss occurs
  - This leads to: Attention mechanisms

Deep Recurrent Neural Networks

- Normal RNN has one hidden layer.
- Deep RNN uses multiple hidden layers.
- Purpose: increase model capacity and representation power.

- Standard RNN hidden state:

$$h_t = f(W_h h_{t-1+} W_x x_t)$$

- At a time
  - Input x_t
  - Hidden state h_t
- Only one layer of transformation

- Deep RNN stacks multiple RNN layers.

Layer 1:

$$h_t^{(1)} = f(W^{(1)} h_{t-1}^{(1)} + U^{(1)} x_t)$$

Layer 2:

$$h_t^{(2)} = f(W^{(2)} h_{t-1}^{(2)} + U^{(2)} h_t^{(1)})$$

So the network becomes **deep in space and time.**

Types of Deep RNN Architectures

- Three common forms:
  - Deep input-to-hidden
    - Multiple layers between input and hidden state.
  - Deep hidden-to-hidden
    - Multiple stacked recurrent layers.
  - Deep hidden-to-output
    - Multiple layers before final output.
- Most practical models use stacked RNN layers.

Advantages of Deep RNNs

- Learn complex sequence patterns
- Capture higher-level temporal features
- Used in
  - speech recognition
  - language modeling
  - machine translation
- Example: Stacked LSTM networks

Recursive Neural Networks

- Recursive neural networks operate on hierarchical structures instead of sequences.
- Input structure: tree structures
- Examples:
  - Syntactic parse trees
  - Mathematical expressions
  - Program structure

- Recursive networks combine child node representations.

If two child nodes are

$$h_1, h_2$$

Parent representation:

$$h_p = f(W[h_1; h_2] + b)$$

The **same weights** are used for all nodes in the tree.

Ex: Sentence Parsing

- Sentence:

```
The movie is very good
```

Parse tree:

```
        Sentence
       /        \
    NP          VP
```

Recursive networks compute **representations bottom-up**.

Words → phrases → sentence.

Applications

- Recursive neural networks used in:
- sentiment analysis
- syntactic parsing
- semantic composition
- program analysis

Ex: Stanford Sentiment Treebank model.

Recurrent NN	Recursive NN
Works on sequences	Works on tree structures
Time-based	Hierarchical
Used in translation	Used in parsing
Ex: Word Sequence	Ex: Parse tree

- Recurrent → sequence (time)
- Recursive → tree (structure)

Exploding and Vanishing Gradients in RNNs

Vanishing & Exploding Gradient Problem In RNN. – Praudyog

- During Backpropagation Through Time (BPTT), gradients are repeatedly multiplied across time steps.

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

- This creates many multiplications of weight matrices and activation derivatives.

- **Vanishing gradient:** Gradients become extremely small during BPTT, preventing learning of long-term dependencies.
- **Exploding gradient:** Gradients grow very large during BPTT, causing unstable weight updates.

Vanishing Gradients

- If the values of derivatives are less than 1, repeated multiplication causes gradients to shrink exponentially.
- Effects
  - Early time steps receive almost zero gradient
  - RNN cannot learn long-term dependencies
- Example:
  - In sentence modeling: “The movie I watched last year was fantastic.”
- The network may forget the word “movie” by the time it predicts fantastic.

Exploding Gradients

- If derivatives are greater than 1, repeated multiplication causes gradients to grow exponentially.
- Effects
  - Extremely large weight updates
  - Training becomes unstable
  - Loss may become NaN

Why RNNs Suffer More

- Unlike feedforward networks, in RNNs:
  - The same weights are applied repeatedly across time
  - Gradients may be multiplied dozens or hundreds of times
- Thus the problem becomes much more severe.

Problem	Solution
Exploding gradients	Gradient Clipping
Vanishing gradients	LSTM, GRU architectures

Long-Term Dependency Problem

- The long-term dependency problem occurs when an RNN is unable to learn relationships between inputs that are far apart in a sequence.
- In many tasks, the prediction at a later time step depends on information that appeared much earlier.
- However, due to vanishing gradients, standard RNNs struggle to preserve this information.
  - Standard RNNs can remember recent information, but often forget information from distant time steps.

Example Sentence

> "I grew up in **France**... I speak fluent _____."

Correct prediction: **French**

In an RNN: Sequence of inputs: $x_1 \to x_2 \to x_3 \to \cdots \to x_T$

The word **France** appears early, but the prediction **French** occurs much later. The RNN must remember the earlier word for many time steps.

Time steps in sequence:

```
France → ... → ... → ... → ... → ... → speak → fluent → ?
```

To predict the final word:

```
Prediction → French
```

The model must **retain information about "France" for many steps.**

- Problem
  - During training, the gradient signal weakens over time, so the network forgets the earlier information.
- In Standard RNN
  - Hidden state gradually loses important information, so the prediction becomes incorrect.

Why Long-Term Dependencies Are Difficult

- During Backpropagation Through Time (BPTT):

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

- If derivatives are small:

$$\left|\frac{\partial h_{k+1}}{\partial h_k}\right|$$

- Then the gradient shrinks exponentially.

- Standard RNNs tend to learn short-term dependencies only.
- Examples where long-term memory is needed:
  - Language modeling
  - Machine translation
  - Speech recognition
  - Text generation

- To address this limitation, special RNN architectures were developed:
  - Long Short-Term Memory (LSTM)
  - Gated Recurrent Unit (GRU)
- These architectures are designed to retain information for long periods.

LSTM: Long Short-Term Memory

Motivation

- Problem with Standard RNNs
  - Vanishing gradients
  - Cannot capture long-term dependencies
  - Important information is forgotten

Example:

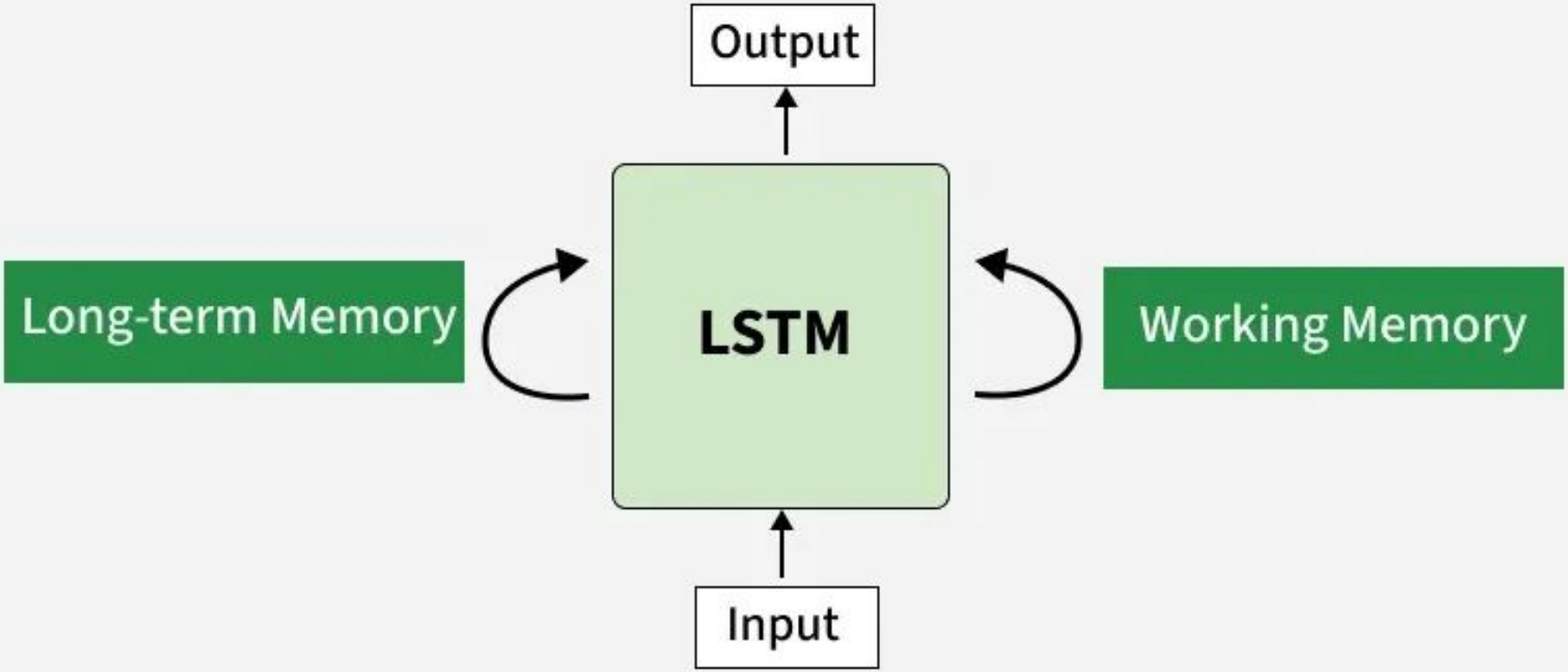
> "I grew up in **France** ... I speak fluent ___"

- Standard RNN struggles to remember **France.**

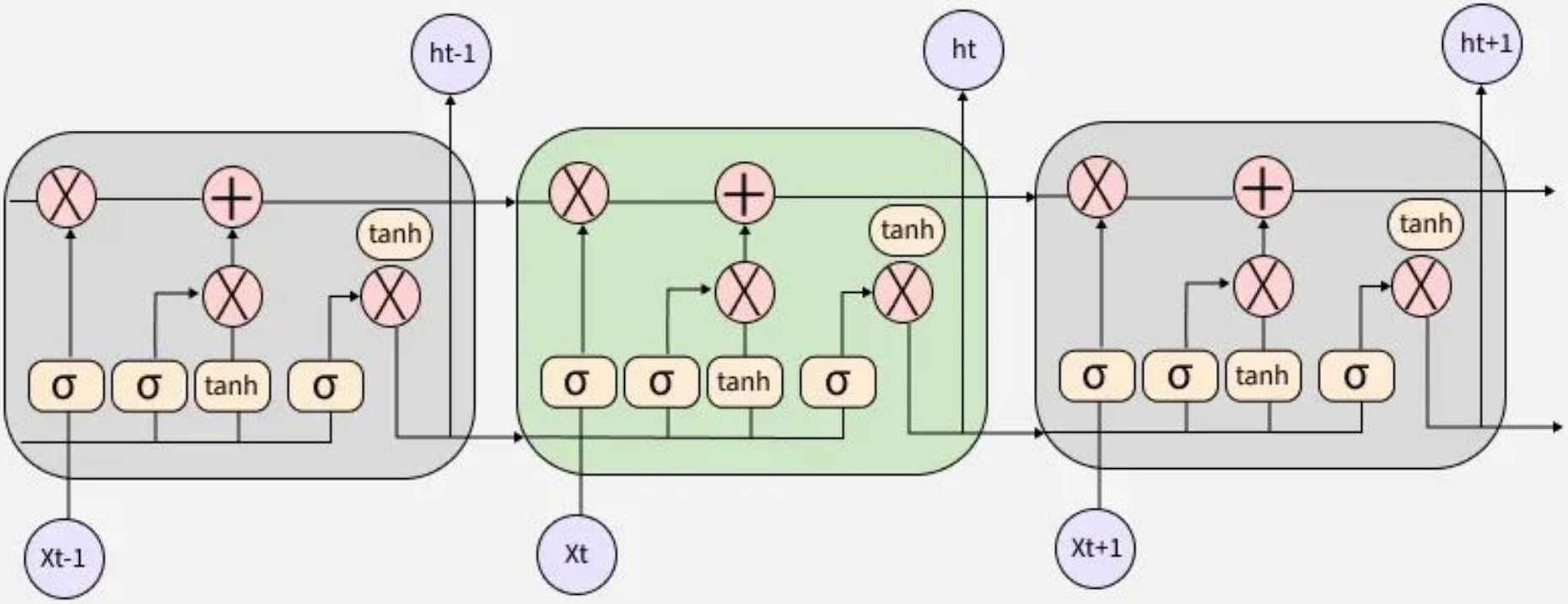
Idea Behind LSTM

- LSTM was proposed to store information for long periods.
- Key idea: Memory Cell
  - Keeps important information
  - Uses gates to control information flow
- LSTM decides:
  - What to remember
  - What to forget
  - What to output
- An LSTM network is composed of a chain of repeating LSTM cells, each containing a cell state (memory) and three gates (forget, input, and output) that regulate information flow.

Introduction To LSTMs

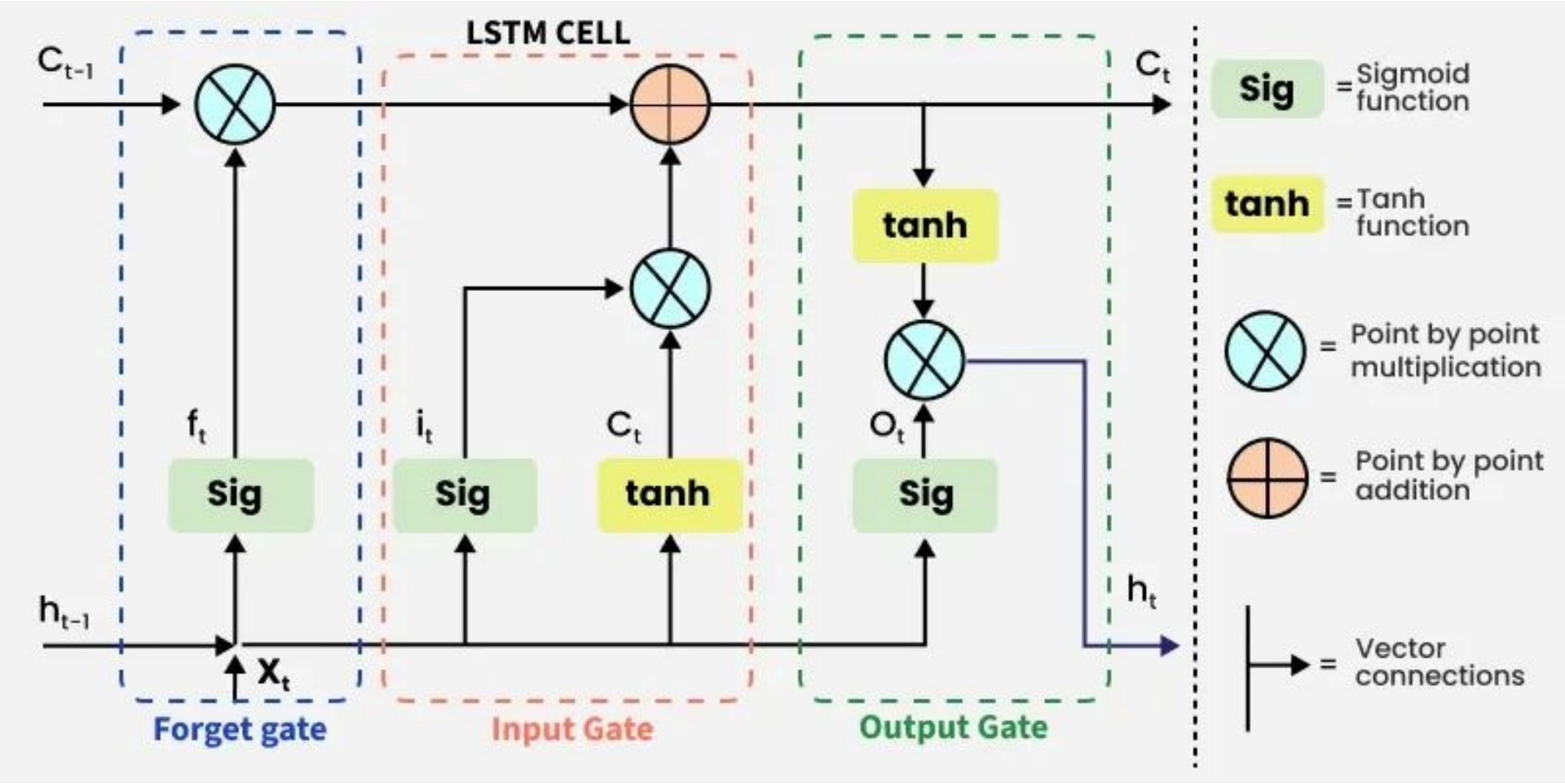


Architecture Of LSTMs



- Main components:
  - Cell state C_t
  - Hidden state h_t
  - Forget gate
  - Input gate
  - Output gate
- Cell state acts like a memory highway.
- Information is retained by the cells and the memory manipulations are done by the gates.

- The memory cell is controlled by three gates:
  - Input gate: Controls what information is added to the memory cell.
  - Forget gate: Determines what information is removed from the memory cell.
  - Output gate: Controls what information is output from the memory cell.
- Allows to selectively retain or discard information flow through the network which allows them to learn long-term dependencies.
- The hidden state is like its short-term memory. This memory is updated using the current input, the previous hidden state and the current state of the memory cell.


LSTM CELL
C_{t-1}
C_t
h_{t-1}
h_t
X_t
f_t
i_t
C_t
O_t
Sig
Sig
tanh
tanh
Sig
Forget gate
Input Gate
Output Gate
Sig = Sigmoid function
tanh = Tanh function
= Point by point multiplication
= Point by point addition
= Vector connections

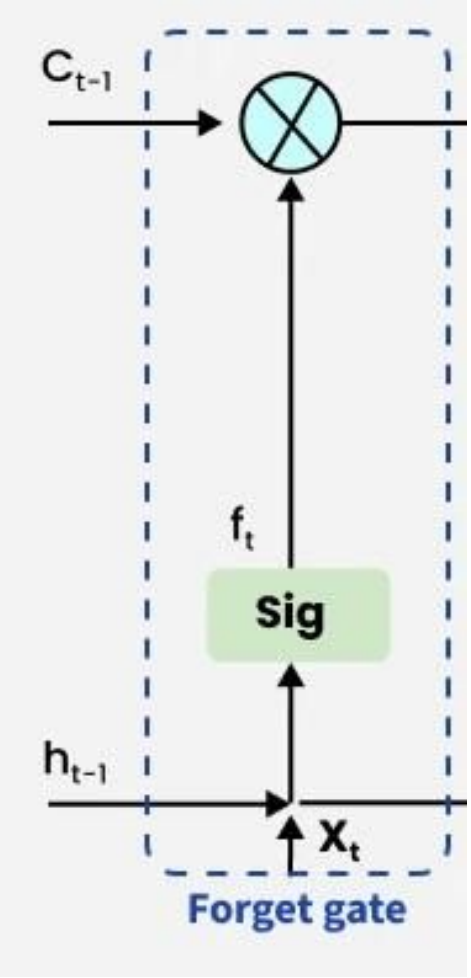
Forget Gate

- Decides what information to discard from memory.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Where

  - f_t = forget gate
  - σ = Sigmoid activation function
  - W_f = weight matrix associated with the forget gate
  - $[h_{t-1}, x_t]$ = concatenation of the current input and the previous hidden state
  - b_f = bias
- If value ≈ 0 → forget
- If value ≈ 1 → keep


C_{t-1}
f_t
Sig
h_{t-1}
X_t
Forget gate

Input gate

- The addition of useful information to the cell state is done by the input gate.
- First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t.
- Then, a vector is created using tanh function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t.
- At last the values of the vector and the regulated values are multiplied to obtain the useful information.

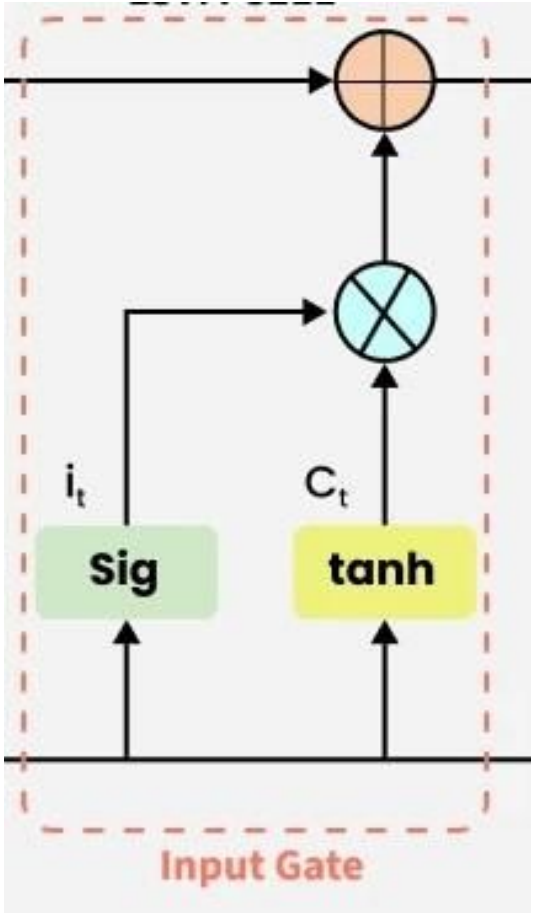
- The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C} = tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

- We multiply the previous state by f_t effectively filtering out the information we had decided to ignore earlier. Then we add $i_t o C_t$ (element wise multiplication) which represents the new candidate values scaled by how much we decided to update each state value.

$$C_t = f_t \circ C_{t-1} + i_t \circ \hat{C}_t$$


i_t
C_t
Sig
tanh
Input Gate

Output gate

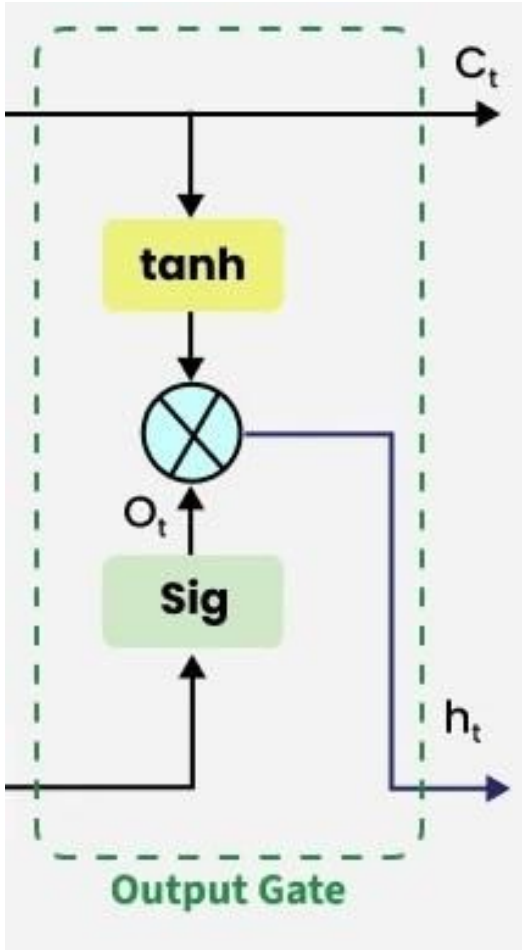
- The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step.
- First, the gate uses a sigmoid function to determine which information from the current cell state will be output.
- This is done using the previous hidden state h_{t-1} and the current input x_t:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- Next, the current cell state C_t is passed through a tanh activation to scale its values between -1 and +1.
- Finally, this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t:

$$h_t = o_t \circ tanh(C_t)$$

- This hidden state h_t is then passed to the next time step and can also be used for generating the output of the network.


C_t
tanh
O_t
Sig
h_t
Output Gate

Complete LSTM Flow

- Steps:
  - Forget irrelevant information
  - Add new candidate information
  - Update cell memory
  - Produce output

Why LSTM Solves Vanishing Gradient

- The cell state provides a nearly linear path for gradients.
- This allows gradients to flow across many time steps without vanishing.
- Thus LSTM can learn long-term dependencies.

Applications of LSTM

- Machine translation
- Speech recognition
- Text generation
- Language modeling
- Video analysis

Advantages of LSTM

- Handles long sequences
- Reduces vanishing gradient problem
- Learns long-term dependencies

Limitations

- More parameters
- Slower training
- Complex architecture

Beam Search

Motivation

- In Artificial Intelligence, many problems require searching through large solution spaces
- Traditional search methods:
  - Depth-First Search (DFS) → low memory but may miss optimal solution
  - Breadth-First Search (BFS) → finds optimal solution but uses high memory
- These methods have limitations in terms of:
  - Efficiency
  - Memory usage

- Beam Search is a heuristic-based search algorithm
- It provides a balance between:
  - Memory consumption
  - Solution optimality
- Instead of exploring all possibilities, Beam Search:
  - Keeps only the top k best candidates (beam width) at each step

Heuristic Techniques

- Heuristic techniques are strategies used to choose the best approach among multiple options
- They use rules or criteria (heuristics) to guide decision-making
- Focus on:
  - Speed and efficiency
  - Rather than exhaustive search

- Does not guarantee optimal solution, but provide good/near-optimal solutions
- Useful for solving complex problems with large search spaces
- Example:
  - Traveling Salesman Problem (TSP)
- Advantage:
  - Reduces computational cost compared to exponential-time algorithms

Beam Search Algorithm

- Beam Search is a heuristic search algorithm used to explore large search spaces
- It expands nodes level by level similar to Breadth-First Search (BFS)
- At each level:
  - Generates all possible successors
  - Evaluates them using a heuristic (score/probability)

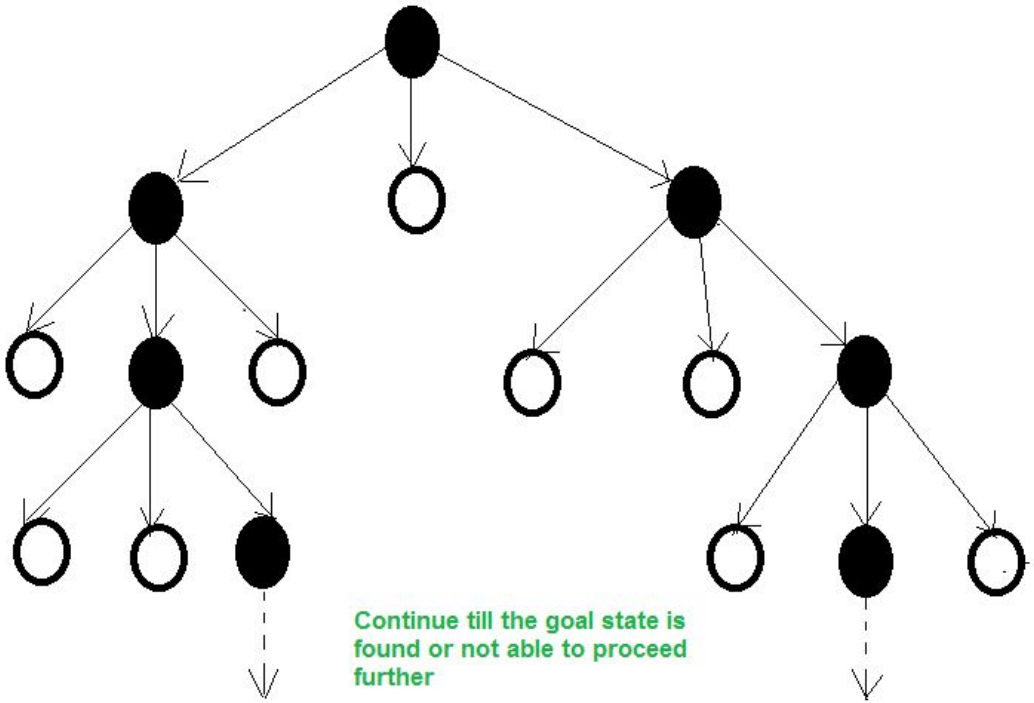
- Instead of keeping all nodes:
  - Retains only the top W best nodes
  - W = beam width
- The process is repeated at each level until the goal is reached
- This limits:
  - Memory usage
  - Computational cost

Characteristics of Beam Search

- Beam Width (W):
  - Number of nodes retained at each level
  - Controls the breadth of the search
- Branching Factor (B):
  - Each node generates B successors
  - Total nodes evaluated at each level = $W \times B$
  - Only top W nodes are selected for next step

- Completeness & Optimality:
  - Not guaranteed to find the optimal solution
  - May discard promising paths due to limited beam width
- Memory Efficiency:
  - Memory usage is limited by beam width W
  - Suitable for resource-constrained environments

- **Example:** Consider a search tree where W = 2 and B = 3. Only two nodes (black nodes) are selected based on their heuristic values for further expansion at each level.



How Beam Search Works

1. **Initialization:**
   a. Start with the root node
   b. Generate its successor nodes
2. **Node Expansion:**
   a. Expand current nodes to generate all possible successors
   b. Evaluate each using a heuristic function
3. **Selection:**
   a. Choose the top W nodes based on heuristic values
   b. These form the next level of search

4. **Iteration:**
   a. Repeat expansion + selection for each level
   b. Continue until goal or stopping condition is met
5. **Termination:**
   a. Stop when:
      i. Goal is reached, or
      ii. No more nodes to expand

LEARN-ONE-RULE Algorithm

- LEARN_ONE_RULE is an algorithm based on Beam Search
- Goal:
  - Learn a single rule that covers a subset of training examples
- Uses a general-to-specific search strategy:
  - Start with a very general rule
  - Gradually make it more specific

- At each step:
  - Generate candidate rules
  - Evaluate them using a performance metric
- Keeps only the best rules (beam width) for further refinement
- Search is greedy:
  - Always selects the most promising rules at each step

Algorithm Execution Flow

1. Start: Initialize the node to Root_Node and Found to False.
2. Search Loop:
   a. If the current node is the goal, set Found to True.
   b. Otherwise, find successors and estimate costs, storing them in the OPEN list.
   c. Continuously select the top W elements from the OPEN list for expansion.
3. Evaluation:
   a. If the goal is found during expansion, return Yes.
   b. If the search concludes without finding the goal, return No.

Advantages of Beam Search

- **Efficiency:** By limiting the number of nodes expanded, Beam Search can navigate large search spaces more efficiently than exhaustive searches.
- **Flexibility:** The algorithm can be adjusted for different problems by modifying the beam width and heuristic function.
- **Scalability:** Suitable for problems where the solution paths are vast and complex, as it does not require all nodes to be stored in memory.

Limitations of Beam Search

- **Suboptimality:** There is no guarantee that Beam Search will find the optimal solution, especially if the beam width is too narrow.
- **Heuristic Dependency:** The effectiveness of Beam Search is highly dependent on the quality of the heuristic function. Poor heuristics can lead to suboptimal searching and results.

Applications of Beam Search Algorithm

- **Natural Language Processing (NLP):** For tasks like machine translation and speech recognition where the goal is to find the best sequence of words or phonemes.
- **Robotics:** In pathfinding algorithms where a robot must find an efficient path in an environment.
- **Game AI:** In strategic games where it is impractical to explore every possible move due to the enormous search space.